



AUTOMATIC MACHINE LEARNING

Tutorial 2 Report: Decision Trees.

Jin Weijian Víctor Chen - 100496723

Teresa Vicente Sánchez - 100496795

Exercise 1: Non ML agent

1. Program an “intelligent” agent using your own rules. To do so, create a method that selects the correct action given the state. Your new agent should be able to reach the treasure while avoiding the obstacles. Is your agent capable of reaching the goal?

This intelligent agent is designed to navigate a custom map in the FrozenLake environment, with the objective of reaching a designated goal (G) while avoiding obstacles like holes (H). The agent selects actions that minimize the distance to the goal at each step. Despite its efforts, it encounters a dead-end scenario that it is unable to solve.

Initial Setup:

- Start Position: The agent begins at the predefined position (11, 10) marked as the start (S) on the map.
- Goal Position: The target is set at (0, 0), where the goal (G) is located.

Action election:

- Movement Directions: The agent evaluates four possible directions: up, down, left, and right.
- Validity Check: It verifies whether each move is within map boundaries and avoids obstacles such as holes (H).
- Distance Calculation: For each valid move, the agent calculates the Manhattan distance from the new position to the goal.
- Greedy Strategy: The move that minimizes this distance is selected as the next action.

Goal-Oriented Movement:

- The agent continuously moves toward the goal, following the action that brings it closer based on the Manhattan distance.
- The process continues until the agent either reaches the goal or encounters obstacles that prevent further progress.

Dead-End Encounter:

- During navigation, the agent reaches a dead end at the point (6, 0), which means it never reaches the goal. This position leads the agent into a loop, where it cannot find a valid path to progress further.

2. The agent you just programmed does not use any machine learning technique. What advantages do you think machine learning may have to control the agent?

- **Improved Decision-Making:**

A Machine Learning based agent can learn from its environment and adapt its behavior over time, which would make it able to overcome obstacles like the dead end we were speaking about.

- **Generalization and Flexibility:**

Machine Learning agents can generalize knowledge across different environments, allowing them to adapt when faced with new challenges. This would work for example if we were to change the map used.

- **Exploration vs. Exploitation Balance:**

These techniques enable a balance between exploring new actions and exploiting known successful ones. This balance encourages the agent to explore less obvious routes, which may lead to finding shortcuts or alternative paths that a purely distance-based strategy would not consider.

- **Experience-Based Learning:**

Unlike the current rule-based agent, an ML agent can learn from past interactions, saving information about successful and unsuccessful paths. The agent becomes better at navigating obstacles over time and leads to more efficient routes to the goal.

Exercise 2: First Decision Tree

1. Create and train a decision tree with the default parameters. Afterward, use the decision tree to predict the actions of the test data. How many actions were correctly predicted? How many were incorrect? What is the precision, recall, and accuracy of the model created?

The decision tree classifier achieved an accuracy of approximately 66.34% on the test set, meaning that the model is correctly predicting the actions based on the state two-thirds of the time. The model has the same recall and a precision of 65.78%

Model Accuracy: 66.34%
Model Precision: 65.78%
Model Recall: 66.34%

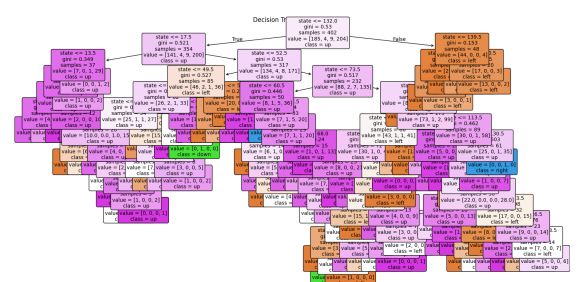
In the excel we have 502 rows of information, since the test set size is 20% of the total data, the number of text samples will be approximately 100. From this we can calculate the number of correct and incorrect predictions.

$$\text{Correct Predictions} = 0.6634 \times 100 = 66.34 \approx 66 \text{ correct predictions}$$

$$\text{Incorrect Predictions} = 100 - 66.34 = 34 \text{ incorrect predictions}$$

2. Obtain the tree generated by the model. Interpret the tree and explain which are the decisions of the model to predict the next action.

This starts with a top-level decision and the condition is that it's ≤ 132 , from here it decides whether to go left or right based on the value of the state. Each subsequent decision leads to a more specific state interval, which further refines the action to be taken.



3. Does this model work better than the one implemented in the previous exercise?

Why? Why not?

The decision tree model (agent2) should outperform the rule-based agent (agent1) due to its learning capabilities, but its actual performance depends on various factors.

Why it may work better:

- The decision tree can learn actions based on actual outcomes and adjust its predictions accordingly.
- With clear metrics to evaluate its performance, it's easier to understand its effectiveness and adjust accordingly.
- Adaptation ability to changing environments without needing manual reprogramming.

Why it might not work better:

- If the training data is insufficient or does not represent the state space adequately, the decision tree may not perform well.
- Decision trees can be prone to overfitting, especially with limited data. If the tree becomes too complex, it might perform poorly on unseen data.
- If not trained properly, the model might still get stuck, similar to the rule-based agent.

Exercise 3: Parameter Tunning

Now we will tune some parameters to try to improve the results from the base model. Since there are a lot of parameters focus only on criterion, splitter, max depth, min samples split and class weight.

Parameter	Value	Description
criterion	'gini'	The function to measure the quality of a split. Gini impurity is used to assess the impurity of the nodes. Alternatively, 'entropy' could be used to measure information gain.
splitter	'best'	The strategy used to choose the split at each node. 'best' chooses the best split, while 'random' selects a random split.
max_depth	10	The maximum depth of the tree. This limits the number of splits and helps to prevent overfitting.
min_samples_split	10	The minimum number of samples required to split an internal node. A higher value prevents the model from splitting too finely on small datasets.
class_weight	'balanced'	Adjusts weights inversely proportional to class frequencies in the input data. This is useful when dealing with imbalanced classes, ensuring that the classifier pays equal attention to all classes.

1. Create and train a decision tree for each parameter combination. Afterward, use the corresponding decision tree to predict the actions of the test data. How many actions were correctly predicted? How many were incorrect? What is the precision, recall, and accuracy of the model created? Use a table to compare these results.

Tree	Criterion	Splitter	Max Depth	Min Samples Split	Class Weight	Precision - Up	Precision - Down	Precision - Left	Precision - Right	Recall - Up	Recall - Down	Recall - Left	Recall - Right	Accuracy
1	Gini	Best	5	10	None	0.00	0.57	0.64	0.00	0.00	0.72	0.55	0.00	0.60
2	Entropy	Best	5	10	None	0.00	0.83	0.59	0.00	0.00	0.35	0.97	0.00	0.64
3	Gini	Random	5	10	None	0.00	0.58	0.53	0.00	0.00	0.62	0.61	0.00	0.58
4	Gini	Best	10	10	None	1.00	0.64	0.79	0.00	0.60	0.84	0.61	0.00	0.70
5	Gini	Best	10	20	None	1.00	0.67	0.81	0.00	0.60	0.84	0.68	0.00	0.74
6	Gini	Best	10	20	Balanced	0.25	0.67	0.74	0.00	0.80	0.62	0.68	0.00	0.64

2. Which set of parameters is better for this problem? Why?

We believe that the best set of parameters are those from tree 5. It has the highest overall accuracy of 0.74, meaning it made the fewest incorrect predictions. It also shows a balanced performance in terms of precision and recall, especially for the "Down" and "Left" actions. For "Down," it achieved 0.67 precision and 0.84 recall, and for "Left," it scored 0.81 precision and 0.68 recall, outperforming most other trees. Tree 5 also performs well for the "Up" action, with perfect precision (1.00), though its recall is slightly lower (0.60), indicating that it accurately identifies "Up" actions but may miss a few.

Tree 4 also performs well, with an accuracy of 0.70 and strong metrics for "Down" and "Left." However, Tree 5 outperforms it in precision for "Left" (0.81 vs. 0.79) and overall accuracy (0.74 vs. 0.70).

3. Obtain the tree generated by the best model. Interpret the tree and explain which are the decisions of the model to predict the next action.

The decision tree uses "state" thresholds to predict the next action based on a series of splits. At the root, if the state is less than or equal to 132, the model splits further, checking values like 17.5 or 13.5. For example, if the state is less than or equal to 1.5, the model predicts action 3 (with weights favoring action 3). Similarly, if the state is less than 36.5, action 0 might be predicted.

When the state is greater than 132, further splits occur, such as comparing to 136.5, which might lead to action 0. This hierarchical decision-making allows the model to navigate the decision space efficiently. *[Table 1]*

4. Does this model work better than those implemented in the previous exercises? Why? Why not?

Model 3 does a better job than Model 2 because it fine-tunes the settings for the decision tree and gives a deeper look at how well the model is doing. It uses different metrics like

precision or recall, which help us understand its performance in more detail. This way, we can see how well the model is actually able to tell apart different actions.

Exercise 4: Deploy the agent

1. Is this model capable of reaching the goal? If not, why not?

No, this model isn't really capable of reaching the goal, and the main reason is the limited training data it had to work with. During training, the agent only learned from a small set of state-action pairs. If those didn't cover enough different scenarios or obstacles in the FrozenLake environment, the model just didn't pick up the skills it needed to navigate effectively.

Take the starting position at row 11, for example. The agent tends to move left without thinking twice, but it doesn't stop when it runs into the hole at (11, 3). Instead, it just keeps jumping into it over and over. This shows that the agent hasn't figured out how to avoid traps, which is a huge deal if you want to make it to the goal.

2. Does this model work better than the one implemented in exercise 1 (the one you implemented)? Why? Why not?

Not at all! This decision tree agent (let's call it agent4) does not perform better than the one from Exercise 1 (we'll call that agent1). In fact, agent4 is doing much worse, mainly because it didn't get enough training data to learn from.

Since the training set was so small, the decision tree didn't encounter enough different situations, so it really struggles when faced with obstacles. You can see this when agent4 often makes moves that lead it into holes or dead ends instead of toward the goal. Both agents tend to go left first, showing they have a similar approach, but agent4 just gets stuck and can't move upward because it's not experienced enough. In contrast, agent1 is able to get closer to the goal at (6, 0) even though it eventually gets trapped in a loop.

Agent1 might have a simple strategy, but at least it uses some basic rules to get around, which helps it navigate better than the decision tree model. So, overall, agent4's struggles are due to not having enough practice, while agent1 does a better job of finding its way, even with its limitations. If agent4 had a larger training set or if the map was smaller but it kept the same data, it could definitely perform better and navigate the FrozenLake more effectively.

Appendix:

Table 1: Decision Tree 5 Thresholds, Predicted Actions, and Weights

Threshold	Predicted Action (Class)	Weights (Action 0, 1, 2, 3)
state <= 1.50	3	[4.00, 0.00, 0.00, 14.00]
state > 1.50	3	[1.00, 0.00, 0.00, 8.00]
state <= 12.50	0	[1.00, 0.00, 0.00, 1.00]
state > 13.50	3	[0.00, 0.00, 1.00, 2.00]
state <= 22.50	0	[1.00, 0.00, 0.00, 0.00]
state <= 24.50	0	[5.00, 0.00, 0.00, 4.00]
state > 24.50	3	[4.00, 0.00, 1.00, 6.00]
state > 31.00	3	[0.00, 0.00, 0.00, 1.00]
state <= 36.50	0	[4.00, 1.00, 0.00, 0.00]
state > 36.50	0	[9.00, 0.00, 0.00, 9.00]
state > 39.50	3	[1.00, 1.00, 0.00, 7.00]
state <= 51.50	0	[11.00, 0.00, 0.00, 3.00]
state > 51.50	0	[7.00, 0.00, 0.00, 0.00]
state <= 60.50	3	[1.00, 0.00, 0.00, 15.00]
state <= 61.50	2	[0.00, 0.00, 3.00, 0.00]
state <= 65.50	3	[6.00, 1.00, 0.00, 7.00]
state > 65.50	3	[1.00, 0.00, 1.00, 13.00]
state <= 76.50	0	[5.00, 0.00, 0.00, 0.00]
state <= 80.00	3	[7.00, 0.00, 0.00, 11.00]
state > 80.00	3	[1.00, 0.00, 0.00, 7.00]
state <= 97.50	0	[16.00, 1.00, 0.00, 13.00]
state <= 104.50	3	[4.00, 0.00, 0.00, 5.00]
state > 104.50	3	[1.00, 0.00, 0.00, 16.00]
state <= 136.50	0	[3.00, 0.00, 0.00, 1.00]
state <= 139.50	0	[14.00, 0.00, 0.00, 0.00]
state <= 140.50	0	[4.00, 0.00, 0.00, 1.00]
state > 140.50	0	[13.00, 0.00, 0.00, 2.00]